OAVS TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ Question Bank

Structures in C — Complete Topic-wise Practice Set

Structures

Homogeneous & Heterogeneous Collections • Declaration & Initialization • Member Access • Arrays of Structures • Nested & Anonymous Structures • Self-Referential Structures • Pointers to Structures • Functions & Structures

222	19	100%
TOTAL MCQS	TOPICS COVERED	EXPLAINED ANSWERS

EXAM-ORIENTED • REVISION-READY • ANSWER KEY INCLUDED

Table of Contents

1	Structure as a Homogeneous and Heterogeneous Collection	14 Questions
2	Applications of Structures	10 Questions
3	Syntax of Declaring Structure	14 Questions
4	Initializing Structures	13 Questions
5	Structure Variables	10 Questions
6	Accessing Structure Members Using Member Operator (.)	13 Questions
7	Arrays of Structures	14 Questions
8	Array as a Member of Structure	12 Questions
9	Conceptual Difference Between Arrays and Structures	12 Questions
10	Functions and Structures	12 Questions
11	Nested Structures	13 Questions
12	Anonymous Structures	9 Questions
13	Self-Referential Structures	10 Questions
14	Pointer as Member of Structure	10 Questions
15	Pointer to Structure	13 Questions
16	Member Selector Operator (->)	12 Questions
17	Comparison Between Indirection (*) Operator and Member Selector (->) Operator	10 Questions
18	Structure as Function Argument	11 Questions
19	Structure as Function Return Type	10 Questions

Structure as a Homogeneous and Heterogeneous Collection

14 Multiple Choice Questions | With Answers & Explanations

Q1 DEFINITION

Which term best defines a C structure?

A. A collection of variables of the same data type
B. A user-defined data type that groups variables of different data types under one name
C. A pointer to a function
D. A named constant

Ans: B

Explanation: A structure groups related variables of possibly different data types under a single name.

Q2 HOMOGENEOUS COLLECTION

An array is best described as a _____ collection of elements.

A. Heterogeneous
B. Homogeneous
C. Nested
D. Anonymous

Ans: B

Explanation: Arrays store elements of the same data type, making them a homogeneous collection.

Q3 HETEROGENEOUS COLLECTION

A structure is best described as a _____ collection of elements.

A. Homogeneous
B. Sequential
C. Heterogeneous
D. Static

Ans: C

Explanation: Structures can hold members of different data types, making them heterogeneous.

Q4 CHARACTERISTICS

Which of the following is a key characteristic of a structure?

A. All members must share the same data type
B. Members are accessed only through numeric indices
C. Members can be of different data types grouped under one name
D. A structure cannot contain more than one member

Ans: C

Explanation: Structures allow grouping of dissimilar data types under a single identifier.

Q5 NEED FOR STRUCTURES

Why are structures needed in C programming?

A. To store only integer values
B. To represent a record containing related but dissimilar data items
C. To replace all arrays in a program
D. To perform arithmetic operations faster

Ans: B

Explanation: Structures let a programmer represent a real-world record, such as a student, as one logical unit.

Q6 ADVANTAGES

Which is an advantage of using structures over separate unrelated variables?

- A They reduce logical grouping of data
- B They allow related data of different types to be handled as a single unit
- C They restrict data to one type only
- D They eliminate the need for functions

Ans: B

Explanation: Structures bundle related data together, simplifying handling and passing of grouped information.

Q7 APPLICATIONS

Which real-world entity is most suitable to represent using a structure?

- A A single integer marks value
- B A list of 10 identical float numbers
- C A student record with name, roll number, and marks
- D A single character grade

Ans: C

Explanation: A student record naturally has mixed data types (string, integer, float), ideal for a structure.

Q8 COMPARISON WITH BASIC DATA TYPES

Unlike basic data types such as int or float, a structure:

- A Can hold only one value
- B Is a derived, user-defined data type that can hold multiple values of possibly different types
- C Cannot be used to declare variables
- D Is always smaller in size than an int

Ans: B

Explanation: A structure is a derived data type composed of one or more basic (or derived) types.

Q9 HOMOGENEOUS VS HETEROGENEOUS

Which statement correctly distinguishes arrays from structures?

- A Arrays are heterogeneous; structures are homogeneous
- B Both are strictly homogeneous
- C Arrays are homogeneous; structures are heterogeneous
- D Both are strictly heterogeneous

Ans: C

Explanation: Arrays hold same-type elements, while structures may hold members of different types.

Q10 CHARACTERISTICS

Which of these is generally NOT true of a plain C structure?

- A It can group variables of different types
- B It automatically converts values between unrelated member types
- C It can be used to declare multiple variables
- D It can contain arrays as members

Ans: B

Explanation: Structures do not automatically convert types between unrelated members; each member retains its own type.

Q11 DEFINITION

Which keyword is used to define a structure in C?

A struct
B record
C class
D union

Ans: A

Explanation: The keyword 'struct' is used to define a structure in C.

Q12 NEED FOR STRUCTURES

Without structures, representing a record with mixed data types in C would require:

A A single array of one basic type
B Multiple unrelated variables that are harder to manage together
C Only pointers with no variables
D Only global constants

Ans: B

Explanation: Without structures, related but dissimilar data must be handled as scattered, separately-named variables.

Q13 APPLICATIONS

Which of the following is NOT a typical application of a structure?

A Representing an employee record
B Representing a complex number (real and imaginary parts)
C Storing a million identical sensor readings of the same type efficiently
D Representing coordinates (x, y) of a point

Ans: C

Explanation: Storing many identical values of one single type is better suited to an array, not a structure.

Q14 COMPARISON WITH BASIC DATA TYPES

A structure variable, unlike a basic-type variable, occupies memory that:

A Is always exactly 4 bytes
B Is the combined layout of all its members, possibly with compiler-added padding
C Is shared with all other structure variables of any type
D Cannot be determined at compile time

Ans: B

Explanation: A structure's size depends on the sizes of its members plus any padding added for alignment.

Applications of Structures

10 Multiple Choice Questions | With Answers & Explanations

Q1 RECORD REPRESENTATION

Which of the following best represents record representation using a structure?

A. int marks[50];
B. float avg;
C. char grade;
D. struct Student { char name[20]; int roll; float marks; };

Ans: D

Explanation: A structure grouping name, roll number and marks into one record exactly represents record representation.

Q2 PRACTICAL APPLICATIONS

In a payroll system, which member combination is most practical for an Employee record?

A. name, employee_id, basic_salary, department
B. Only basic_salary repeated 100 times
C. Only employee_id as an array
D. A single float variable

Ans: A

Explanation: A payroll record naturally groups name, id, salary and department of different types into one structure.

Q3 DATA ORGANIZATION

Structures help in data organization mainly by:

A. Forcing all data to be integers
B. Removing the need for variable names
C. Logically grouping related fields under one named unit
D. Making all variables global

Ans: C

Explanation: Structures organize data by logically grouping related fields together under a single name.

Q4 REAL-WORLD USAGE

Which real-world system is a classic use case for structures?

A. Storing a single temperature reading
B. A library system storing book title, author, ISBN, and price together
C. Storing one boolean flag
D. Storing a single character

Ans: B

Explanation: A library book record has multiple related fields of different types, ideal for a structure.

Q5 ADVANTAGES

Which is a direct advantage of using structures for record representation?

A. It increases the number of unrelated variables needed
B. It prevents grouping of any data
C. It requires re-declaring each field separately every time it is used
D. It allows a whole record to be passed to a function using a single variable

Ans: D

Explanation: Because all fields are bundled together, an entire record can be handled using just one structure variable.

Q6 PRACTICAL APPLICATIONS

A structure representing a bank account would most appropriately include which members?

- A account_number, holder_name, balance
- B Only 100 integers of account numbers
- C A single char
- D Only balance repeated for every customer in one variable

Ans: A

Explanation: A bank account record naturally combines an account number, holder name and balance of different types.

Q7 DATA ORGANIZATION

Which of these is a poor use of a structure, better handled by an array instead?

- A Employee record with id, name, and salary
- B Point with x and y coordinates
- C Storing 1000 identical exam scores of the same type
- D Book record with title, author, price

Ans: C

Explanation: A collection of many identical-type values is homogeneous data, better suited to an array than a structure.

Q8 REAL-WORLD USAGE

Structures are commonly used in real-world C programs for all of the following EXCEPT:

- A Representing a date as day, month, and year
- B Performing pure arithmetic like adding two plain integers
- C Representing a rectangle using length and breadth
- D Representing a complex number

Ans: B

Explanation: Simple arithmetic on plain integers does not need a structure; the other options combine related fields.

Q9 ADVANTAGES

One advantage of structures in data organization is that they:

- A Force every member to occupy the same memory address
- B Disallow use inside functions
- C Only work with a single data type
- D Improve code readability by naming related fields together

Ans: D

Explanation: Naming related fields together, such as name, roll and marks, makes code more readable and maintainable.

Q10 RECORD REPRESENTATION

Which is the most accurate description of record representation using structures?

- A Modeling a real-world entity as one unit containing its distinct attributes
- B Storing only one attribute per program
- C Avoiding the use of any data type
- D Replacing all functions in a program

Ans: A

Explanation: Record representation means modeling an entity, like a student or employee, as a single unit of its attributes.

Syntax of Declaring Structure

14 Multiple Choice Questions | With Answers & Explanations

Q1 STRUCTURE DECLARATION SYNTAX

What is the correct general syntax to declare a structure in C?

A struct { tag type member; };
B tag struct type member;
C struct tag member type { };
D struct tag { type member1; type member2; ... };

Ans: D

Explanation: The general syntax is 'struct tag { type member1; type member2; ... };' with a terminating semicolon.

Q2 STRUCTURE TAG

In 'struct Student { ... };', the identifier 'Student' is called the:

A Structure tag
B Structure variable
C Member name
D Function name

Ans: A

Explanation: 'Student' here is the structure tag, naming the structure type.

Q3 MEMBER DECLARATION

Which of the following is a valid member declaration inside a structure?

A int 1number;
B float default;
C int rollNumber;
D char struct;

Ans: C

Explanation: 'rollNumber' is a valid identifier; the other options violate naming rules or use reserved keywords.

Q4 VALID AND INVALID DECLARATIONS

Which structure declaration is syntactically INVALID in C?

A struct Point { int x; int y; };
B struct Point { int x; int y; }
C struct { int x; int y; } p1;
D struct Point { int x, y; };

Ans: B

Explanation: A structure definition must end with a semicolon; option B is missing the terminating semicolon.

Q5 STRUCTURE DEFINITION

A structure definition in C specifies:

A Only the size of the structure
B Only the tag name with no members
C The values stored in the structure
D The template listing the members and their types

Ans: D

Explanation: A structure definition is a blueprint describing member names and types; it does not allocate storage by itself.

Q6 NAMING CONVENTIONS

Which of the following is an invalid structure tag name?

A 2Dpoint
B Point2D
C _Employee
D StudentRecord

Ans: A

Explanation: Identifiers cannot begin with a digit, so '2Dpoint' is invalid as a structure tag.

Q7 STRUCTURE DECLARATION SYNTAX

In 'struct Employee { int id; float salary; } e1;', what does 'e1' represent?

A A member of the structure
B The structure tag
C A structure variable declared along with the definition
D A data type keyword

Ans: C

Explanation: 'e1' is a variable of type 'struct Employee', declared at the same time as the structure definition.

Q8 MEMBER DECLARATION

Which statement about structure members is TRUE?

A All members must have unique data types across the whole program
B Members can be of different data types, including arrays and pointers
C A structure can have at most two members
D Members must always be initialized inside the declaration

Ans: B

Explanation: Structure members may be of any valid type, including arrays, pointers, and even other structures.

Q9 VALID AND INVALID DECLARATIONS

Which of these correctly declares an anonymous (untagged) structure variable?

A struct { int x; };
B struct Point x;
C struct Point { int x; };
D struct { int x; } p;

Ans: D

Explanation: Omitting the tag but including a variable name, as in 'struct { int x; } p;', declares an untagged structure variable.

Q10 STRUCTURE DECLARATION SYNTAX

Which of the following correctly declares a structure with an integer and a character member?

A struct Item { int code; char grade; };
B struct Item (int code; char grade;);
C struct Item [int code; char grade;];
D struct Item int code; char grade;

Ans: A

Explanation: Curly braces enclose the member list, followed by a semicolon after the closing brace.

Q11 NAMING CONVENTIONS

Which is a recommended good practice while naming a structure tag?

A Always use a C reserved keyword
B Start the tag name with a digit
C Use a meaningful, descriptive name such as 'Employee' or 'Point'
D Always use single non-descriptive letters like 'x'

Ans: C

Explanation: Meaningful tag names improve code readability and maintainability.

Q12 STRUCTURE TAG

Once a structure tag such as 'struct Book' is defined, it can be used to:

A Declare a new keyword in C
B Declare further variables of that structure type elsewhere in the program
C Automatically create global variables
D Replace the 'int' data type

Ans: B

Explanation: The tag acts as a type name, allowing declaration of additional variables of that structure type later.

Q13 MEMBER DECLARATION

Can a structure member itself be an array, such as 'char name[20];' inside a structure?

A No, arrays are never allowed as members
B Only if the array is of type int
C No, only pointers are allowed as members
D Yes, arrays are valid structure members

Ans: D

Explanation: Arrays are perfectly valid as structure members, for example to store a string like a name.

Q14 VALID AND INVALID DECLARATIONS

Which structure declaration correctly nests one structure inside another?

A struct Date { int d, m, y; }; struct Employee { char name[20]; struct Date doj; };
B struct Employee { char name[20]; Date doj; };
C struct Employee { struct Date; };
D struct Employee { int Date doj; };

Ans: A

Explanation: To nest structures, the inner structure must be declared first and referenced using 'struct Date doj;' inside the outer one.

Initializing Structures

13 Multiple Choice Questions | With Answers & Explanations

Q1 INITIALIZATION SYNTAX

Which is the correct syntax to initialize a structure variable at the time of declaration?

A struct Point p; p = {1,2};
B struct Point p(1,2);
C struct Point p[1,2];
D struct Point p = {1, 2};

Ans: D

Explanation: A structure variable can be initialized at declaration using 'struct Point p = {1, 2};', assigning values to members in order.

Q2 MEMBER INITIALIZATION

In 'struct Point p = {5, 10};' for 'struct Point { int x; int y; };', what happens?

A p.x is set to 5 and p.y is set to 10, in declaration order
B Both p.x and p.y are set to 5
C Only p.y is initialized
D It causes a compile-time error

Ans: A

Explanation: Initializer values are assigned to members in the order the members are declared.

Q3 PARTIAL INITIALIZATION

What happens when a structure is partially initialized, e.g. 'struct Point p = {5};'?

A It is always a compile error
B Both members get value 5
C The remaining uninitialized members are automatically set to zero
D The remaining members contain the previous value of p

Ans: C

Explanation: In partial initialization, the specified members get the given values while the rest are automatically set to zero.

Q4 COMPLETE INITIALIZATION

'struct Emp { int id; float salary; } e = {101, 25000.0};' is an example of:

A Partial initialization
B Complete initialization
C Invalid initialization
D Runtime initialization only

Ans: B

Explanation: All members (id and salary) are given explicit values, making this a complete initialization.

Q5 INITIALIZATION RULES

Which rule applies to structure initialization in C?

A Initializer values may appear in any random order regardless of member order
B A structure can never be initialized at declaration
C Only the last member can be initialized
D Initializer values are assigned to members strictly in their declared order

Ans: D

Explanation: In standard positional initialization, values map to members in the exact order they were declared.

Q6 INITIALIZATION SYNTAX

Which C99 feature allows initializing specific structure members by name?

A Designated initializers, e.g. {.roll = 5, .name = "Amit"}
B Anonymous unions
C Bit-fields
D Typedef aliasing

Ans: A

Explanation: C99 designated initializers let you initialize specific members by name, using the '.member = value' syntax.

Q7 MEMBER INITIALIZATION

Can individual members be assigned values after declaration instead of during initialization?

A No, only during declaration
B Only for the first member
C Yes, using the member access operator, e.g. p.x = 5;
D Only using pointers

Ans: C

Explanation: Members can always be assigned values after declaration using the dot operator, e.g. 'p.x = 5;'.

Q8 PARTIAL INITIALIZATION

If 'struct Rect { int len, wid; } r = {10};' is written, what is the value of r.wid?

A Garbage/undefined value
B 0
C 10
D It causes a syntax error

Ans: B

Explanation: With partial initialization, unspecified trailing members are automatically initialized to zero.

Q9 INITIALIZATION RULES

Which of the following initializations is INVALID in standard C?

A struct Point p = {1, 2};
B struct Point p = {0};
C struct Point p = {.x = 1, .y = 2};
D struct Point p = {1, 2, 3, 4}; // struct has only 2 members

Ans: D

Explanation: Supplying more initializer values than the number of declared members is invalid.

Q10 COMPLETE INITIALIZATION

Complete initialization of a structure means:

A Every member is explicitly assigned a value in the initializer list
B Only the first member is assigned a value
C No member is assigned a value
D The structure is initialized using a loop only

Ans: A

Explanation: Complete initialization provides an explicit value for each and every member of the structure.

Q11 INITIALIZATION SYNTAX

Array members inside a structure, such as 'char name[10];', can be initialized as:

- A name = "Ravi"; only outside of declaration
- B They can never be given initial values
- C struct Student s = {"Ravi", 21}; assigning "Ravi" to the name array member
- D Only using pointer arithmetic

Ans: C

Explanation: A character array member can be initialized with a string literal within the structure's initializer list.

Q12 MEMBER INITIALIZATION

What is the value of an uninitialized local structure variable's members (no initializer given)?

- A Always zero automatically
- B Indeterminate/garbage values
- C Always equal to their previous declaration
- D A compile-time error occurs

Ans: B

Explanation: If a local structure variable is declared without any initializer, its members contain indeterminate values.

Q13 INITIALIZATION RULES

Which is TRUE regarding nested structure initialization, e.g. Employee containing a Date member?

- A Nested members cannot ever be initialized
- B Only the outer structure's first member can be initialized
- C Braces cannot be nested in initializer lists
- D Nested braces can be used to initialize the inner structure's members, e.g. {"Ravi", {12, 5, 2024}}

Ans: D

Explanation: Nested initializer lists using additional braces let you initialize the members of an inner (nested) structure.

Structure Variables

10 Multiple Choice Questions | With Answers & Explanations

Q1 DECLARATION

Which statement correctly declares a structure variable after 'struct Point' has already been defined?

A. struct Point p1;
B. Point p1;
C. struct p1 Point;
D. int Point p1;

Ans: A

Explanation: After a structure type is defined, variables are declared using 'struct TagName variableName;'.

Q2 DEFINITION

A structure definition alone, without declaring any variable, does:

A. Allocate memory for a structure variable immediately
B. Create multiple copies of the structure
C. Not allocate any memory; it only creates a template/type
D. Automatically initialize all future variables to zero

Ans: C

Explanation: Defining a structure only creates a type/template; memory is allocated only when a variable of that type is declared.

Q3 MEMORY ALLOCATION

When is memory actually allocated for a structure?

A. At the time the structure is defined, before any variable exists
B. When a structure variable is declared
C. Only when the program is compiled, regardless of variables
D. Structures never occupy memory

Ans: B

Explanation: Memory is reserved only when a variable of the structure type is declared, not merely when the type is defined.

Q4 ACCESS

How is an individual member of a structure variable accessed?

A. Using square brackets, e.g. p[0]
B. Using the -> operator only, even for non-pointer variables
C. Using the & operator alone
D. Using the dot (.) operator, e.g. p.x

Ans: D

Explanation: For a direct (non-pointer) structure variable, members are accessed using the dot operator.

Q5 USAGE

Multiple structure variables of the same type can be declared as:

A. struct Point p1, p2, p3;
B. struct Point p1; struct Point2 p2;
C. Only one variable per structure type is allowed
D. struct Point p1 = p2 = p3;

Ans: A

Explanation: Multiple variables of the same structure type can be declared together, separated by commas.

Q6 MEMORY ALLOCATION

If 'struct Point { int x; int y; };' and int is 4 bytes, the size of a struct Point variable is typically:

A Always exactly 1 byte
B Undeterminable at compile time
C At least 8 bytes, possibly more with padding
D Always exactly 2 bytes

Ans: C

Explanation: Two int members of 4 bytes each require at least 8 bytes, though padding or alignment may add more.

Q7 DECLARATION

Which of these declares a structure variable at the SAME time as defining the structure?

A struct Point p1; (defined elsewhere)
B struct Point { int x, y; } p1;
C Point p1; struct { int x; };
D p1 struct Point;

Ans: B

Explanation: Placing a variable name right after the closing brace declares a variable at the same time the structure is defined.

Q8 USAGE

Two different structure variables of the same type, p1 and p2, are:

A Forced to always hold identical values
B Not allowed to coexist in the same program
C Sharing the same memory location
D Independent variables, each with its own separate copy of the members

Ans: D

Explanation: Each structure variable has its own distinct memory and copy of member values, independent of the other.

Q9 ACCESS

Given 'struct Point p1;', which is the correct way to assign 5 to its member x?

A p1.x = 5;
B p1->x = 5;
C x.p1 = 5;
D *p1.x = 5;

Ans: A

Explanation: For a plain (non-pointer) structure variable, the dot operator is used: p1.x = 5;.

Q10 DEFINITION

What is the relationship between a structure definition and a structure variable?

A They are the same thing with no distinction
B A variable can exist without any corresponding definition
C The definition provides the type/template, and the variable is an instance of that type occupying memory
D A definition can only be used once in an entire program

Ans: C

Explanation: The definition is the type blueprint; a variable is an actual instance of that type that occupies memory.

Accessing Structure Members Using Member Operator (.)

13 Multiple Choice Questions | With Answers & Explanations

Q1 MEMBER ACCESS OPERATOR

Which operator is used to access a member of a structure variable directly (not through a pointer)?

A ->
B .
C ::
D &

Ans: B

Explanation: The dot (.) operator accesses members of a structure variable directly.

Q2 SYNTAX

The correct syntax to access member 'roll' of structure variable 's' is:

A s->roll
B s[roll]
C roll.s
D s.roll

Ans: D

Explanation: The syntax 'variable.member' is used to access a structure member directly.

Q3 ACCESSING MEMBERS

Given 'struct Point p = {3, 4};', which expression correctly reads the value of p's y member?

A p.y
B p[y]
C p->y (invalid here since p is not a pointer)
D y.p

Ans: A

Explanation: 'p.y' correctly reads member y of the non-pointer structure variable p.

Q4 READING VALUES

Which line correctly prints the 'salary' member of structure variable 'emp' using printf?

A printf("%f", emp);
B printf("%f", &emp.salary);
C printf("%f", emp.salary);
D printf("%f", salary.emp);

Ans: C

Explanation: 'emp.salary' correctly accesses the value of the salary member for use in printf.

Q5 MODIFYING VALUES

Which statement correctly modifies the 'marks' member of structure variable 's' to 95?

A marks = 95.s;
B s.marks = 95;
C s->marks = 95; (only valid if s were a pointer)
D 95 = s.marks;

Ans: B

Explanation: The dot operator on the left side of an assignment correctly modifies the member's value: s.marks = 95;.

Q6 APPLICATIONS

The member access operator (.) is typically used for which purpose?

A To allocate memory dynamically
B To declare a new structure type
C To compare two entire structures
D To read or modify individual fields of a structure variable

Ans: D

Explanation: The dot operator is used to both read and modify individual member fields of a structure variable.

Q7 MEMBER ACCESS OPERATOR

Can the dot operator be used directly on a structure pointer, e.g. 'ptr.member'?

A No, using dot directly on a pointer type is incorrect; -> or (*ptr).member should be used
B Yes, always with identical behavior to ->
C Yes, but only for integer members
D Yes, but only inside main()

Ans: A

Explanation: The dot operator works on structure variables, not pointers; for pointers, -> or (*ptr).member is required.

Q8 SYNTAX

In 'employee.department.name', what does this expression represent?

A An invalid expression, dot cannot be chained
B Accessing 'employee' as an array
C Accessing the 'name' member of a nested structure member called 'department' inside 'employee'
D Multiplying employee by department by name

Ans: C

Explanation: Chained dot operators access members of nested structures, here reaching 'name' inside 'department' inside 'employee'.

Q9 READING VALUES

Which of these correctly stores the value of s.marks into a variable 'm'?

A m.s = marks;
B int m = s.marks;
C s.marks = m; (this stores m into s.marks, not the reverse)
D m = marks.s;

Ans: B

Explanation: 'int m = s.marks;' reads the value stored in s.marks and assigns it to m.

Q10 MODIFYING VALUES

After executing 's.age = s.age + 1;', what happens?

A A new structure variable is created
B Nothing changes, this is a syntax error
C The structure tag is renamed
D The 'age' member of 's' is incremented by 1

Ans: D

Explanation: This reads the current value of s.age, adds 1, and stores the result back into s.age.

Q11 APPLICATIONS

Which scenario demonstrates a correct application of the member access operator?

A Copying one member value to another variable of the same type, e.g. int r = s.roll;

B Declaring a new structure tag

C Allocating memory using malloc for a structure

D Comparing two structures using the == operator directly

Ans: A

> **Explanation:** The dot operator is correctly used to copy or read an individual member's value into another variable.

Q12 ACCESSING MEMBERS

For 'struct Book { char title[30]; float price; } b;', which line correctly assigns "C Programming" to title?

A b.title = "C Programming";

B title.b = "C Programming";

C strcpy(b.title, "C Programming");

D b->title = "C Programming";

Ans: C

> **Explanation:** Since 'title' is a character array, string assignment requires strcpy(); direct '=' assignment is not allowed for arrays.

Q13 MEMBER ACCESS OPERATOR

Which of these is TRUE about the dot (.) operator's precedence?

A It has lower precedence than arithmetic operators

B It has very high precedence, binding tightly to the variable and member name

C It has the lowest precedence among all C operators

D It cannot be used in expressions with other operators

Ans: B

> **Explanation:** The dot operator has very high precedence, so it is evaluated before most other operators in an expression.

Arrays of Structures

14 Multiple Choice Questions | With Answers & Explanations

Q1 DECLARATION

Which declares an array of 50 structures of type 'struct Student'?

A. struct Student[50] arr;
B. struct arr Student[50];
C. Student struct arr[50];
D. struct Student arr[50];

Ans: D

Explanation: The correct syntax places the array size after the variable name: struct Student arr[50];.

Q2 INITIALIZATION

Which correctly initializes an array of 2 structures, e.g. struct Point arr[2]?

A. struct Point arr[2] = {{1,2}, {3,4}};
B. struct Point arr[2] = {1,2,3,4}; using nested braces is required, this form is not preferred
C. struct Point arr = {{1,2},{3,4}};
D. struct Point arr[2] = (1,2)(3,4);

Ans: A

Explanation: Each element's initializer is wrapped in its own braces within the outer braces: {{1,2},{3,4}}.

Q3 ACCESSING ELEMENTS

How do you access the 'name' member of the 3rd element of 'struct Student arr[10];'?

A. arr.name[2]
B. arr[name].2
C. arr[2].name
D. arr->2.name

Ans: C

Explanation: Array indexing is applied first, then the dot operator accesses the member: arr[2].name.

Q4 APPLICATIONS

Arrays of structures are most useful for:

A. Storing a single scalar value
B. Managing a collection of records of the same type, like a class of students
C. Replacing all pointers in a program
D. Avoiding the use of loops entirely

Ans: B

Explanation: An array of structures is ideal for holding many records of the same type, such as a list of students.

Q5 CHARACTERISTICS

Which characteristic is TRUE for an array of structures?

A. Each element can be of a different structure type
B. All elements must have different member names
C. Only the first element can be accessed
D. All elements share the same structure type and member layout

Ans: D

Explanation: Every element in an array of structures is of the identical structure type and layout.

Q6 DECLARATION

Given 'struct Point { int x, y; };', which correctly declares an uninitialized array of 5 such structures?

A struct Point pts[5];
B struct Point pts(5);
C Point pts[5];
D struct Point[5] pts;

Ans: A

Explanation: 'struct Point pts[5];' declares an uninitialized array of 5 structure elements.

Q7 ACCESSING ELEMENTS

Which loop correctly prints the roll number of every element in 'struct Student s[10];'?

A for(i=0;i<10;i++) printf("%d", s.roll[i]);
B for(i=0;i<10;i++) printf("%d", s[i]->roll);
C for(i=0;i<10;i++) printf("%d", s[i].roll);
D for(i=0;i<10;i++) printf("%d", roll[i].s);

Ans: C

Explanation: Each array element is accessed with s[i], then its member with the dot operator: s[i].roll.

Q8 INITIALIZATION

What value does arr[2].x hold if 'struct Point arr[3] = {{1,1},{2,2}};' (3rd element not given)?

A Garbage value only, x is undefined
B 0, since the 3rd element is not explicitly initialized
C Same as arr[1].x
D It causes a compile-time error

Ans: B

Explanation: An array of structures follows the same partial-initialization rule; missing elements/members default to zero.

Q9 APPLICATIONS

Which is a practical example of using an array of structures?

A A single float variable holding one temperature
B An int variable storing one student's roll number
C A single character variable
D An array 'struct Employee emp[100];' to store data of 100 employees

Ans: D

Explanation: An array of structures naturally stores many employee records, each with its own set of fields.

Q10 CHARACTERISTICS

What is the memory layout of an array of structures in C?

A Elements are stored in contiguous memory, one structure's block right after another
B Each element is stored in a randomly scattered location
C Only the first element is stored in memory; others are references
D Structures cannot form arrays at all

Ans: A

Explanation: Like any array, an array of structures is stored contiguously in memory, one structure block after another.

Q11 DECLARATION

Which is a valid way to declare and partially initialize a 4-element array of structures, giving values to only the first 2?

A struct Point arr[4] = {1,2,3,4,5,6,7,8};

B struct Point arr[] = ; (no braces)

C struct Point arr[4] = {{1,2},{3,4}};

D struct Point arr(4) = {{1,2},{3,4}};

Ans: C

Explanation: Providing initializer braces for only the first two elements leaves the remaining elements zero-initialized.

Q12 ACCESSING ELEMENTS

To modify the marks of the 5th student in 'struct Student s[20];' to 88, you would write:

A s.marks[5] = 88;

B s[4].marks = 88;

C s[5].marks = 88; (this modifies the 6th student)

D marks[4].s = 88;

Ans: B

Explanation: Since array indices start at 0, the 5th student is at index 4: s[4].marks = 88;.

Q13 APPLICATIONS

Which built-in C mechanism is typically combined with an array of structures to process all records?

A The union keyword

B Recursion only

C The switch statement exclusively

D A for or while loop iterating over the array indices

Ans: D

Explanation: Loops are the standard way to iterate through and process every element of an array of structures.

Q14 CHARACTERISTICS

What happens if you access an out-of-bounds index of an array of structures, e.g. arr[10] for a 10-element array (valid 0-9)?

A It results in undefined behavior since C does not perform automatic bounds checking

B The compiler automatically prevents it at compile time

C It safely returns a zero-initialized structure

D It throws a runtime exception automatically

Ans: A

Explanation: C does not perform automatic array bounds checking, so accessing an out-of-bounds index leads to undefined behavior.

Array as a Member of Structure

12 Multiple Choice Questions | With Answers & Explanations

Q1 DECLARATION

Which correctly declares a structure with an array member for storing 5 subject marks?

A struct Student { int marks; [5]; };
B struct Student { array int marks[5]; };
C struct Student { int marks[5]; };
D struct Student { int[5] marks; };

Ans: C

Explanation: The array member is declared just like a normal array, inside the structure: int marks[5];.

Q2 INITIALIZATION

How can the array member 'marks[3]' inside a structure be initialized at declaration?

A It cannot be initialized inside the structure at all
B struct Student s = {"Ravi", {90, 85, 88}};
C struct Student s = 90,85,88;
D s.marks = {90,85,88}; after declaration

Ans: B

Explanation: Nested braces are used to initialize an array member along with the rest of the structure's initializer list.

Q3 ACCESS

How do you access the 2nd mark (index 1) stored in the array member 'marks' of structure variable 's'?

A s[1].marks
B marks[1].s
C s.marks.1
D s.marks[1]

Ans: D

Explanation: First the dot operator reaches the array member, then normal indexing accesses its element: s.marks[1].

Q4 APPLICATIONS

An array as a structure member is most useful for:

A Storing multiple related values of the same type for one record, like 5 subject marks for one student
B Storing values of different types together
C Replacing the entire structure with an array
D Declaring function prototypes

Ans: A

Explanation: An array member stores several same-type values, like marks in multiple subjects, belonging to a single record.

Q5 MEMORY ORGANIZATION

In 'struct Student { char name[20]; int marks[5]; };', how is memory organized?

A name and marks share the exact same memory address
B Only name occupies memory; marks is ignored
C name's 20 bytes and marks's memory are laid out sequentially within the structure (subject to padding)
D marks is stored in a completely separate structure automatically

Ans: C

Explanation: Array members occupy their designated space sequentially within the structure's overall memory layout.

Q6 DECLARATION

Which structure below correctly stores a name and five test scores?

A struct Result { char name; int scores[5]; };

B struct Result { char name[20]; int scores[5]; };

C struct Result { char name[20]; int scores; };

D struct Result { char[20] name; int scores[5]; };

Ans: B

Explanation: Both members are correctly declared as arrays: char name[20] for a string, int scores[5] for five scores.

Q7 INITIALIZATION

Which of the following partially initializes the array member, leaving the rest as zero?

A struct Item it = ; (empty)

B struct Item it = {1,2,3,4,5,6,7,8,9,10}; regardless of the array's actual size

C This can never be done for array members

D struct Item { int codes[5]; } it = {{1,2}};

Ans: D

Explanation: Supplying fewer values than the array size zero-initializes the remaining elements of the array member.

Q8 ACCESS

To find the sum of all elements in the array member 'marks[5]' of structure variable 's', you would typically use:

A A loop: for(i=0;i<5;i++) sum += s.marks[i];

B sum = s.marks; directly, since arrays sum automatically

C sum = s[marks].sum;

D A single statement without any loop, since arrays sum themselves

Ans: A

Explanation: A loop is needed to iterate over each element of the array member and accumulate their sum.

Q9 APPLICATIONS

Which scenario best demonstrates 'array as a member of structure'?

A struct Point { int x; int y; };

B struct Flag { int isActive; };

C struct Matrix { int data[3][3]; }; storing a 3x3 grid of numbers for one matrix record

D int arr[10]; declared outside any structure

Ans: C

Explanation: The Matrix structure uses a 2D array member to store all the numbers belonging to a single matrix record.

Q10 MEMORY ORGANIZATION

If 'int' is 4 bytes, how much memory does the array member 'int marks[5];' contribute within a structure (ignoring padding)?

A 5 bytes

B 20 bytes

C 4 bytes

D 1 byte

Ans: B

Explanation: Five int elements of 4 bytes each require 5 x 4 = 20 bytes for that array member.

Q11 DECLARATION

Can the size of an array member inside a structure be specified using a non-constant variable at compile time?

- **A** Yes, always freely using any variable
- **B** Only in C++ but never in standard C
- **C** Yes, but only for character arrays
- **D** No, in standard C the array member's size must generally be a compile-time constant

Ans: D

> **Explanation:** Array members in a standard C structure require a fixed, compile-time constant size (excluding flexible array members).

Q12 ACCESS

Given 'struct Class { char students[10][20]; };', how would you access the name of the 3rd student?

- **A** c.students[2]
- **B** c[2].students
- **C** students.c[2]
- **D** c.students(2)

Ans: A

> **Explanation:** 'students' is a 2D char array member; indexing with [2] accesses the 3rd row (student): c.students[2].

Conceptual Difference Between Arrays and Structures

12 Multiple Choice Questions | With Answers & Explanations

Q1 SIMILARITIES

What is a similarity between arrays and structures?

A. Both must contain data of different types
B. Both cannot be passed to functions
C. Both are derived data types that group multiple elements/members under one name
D. Both allocate memory only at runtime using malloc

Ans: C

Explanation: Arrays and structures are both derived data types that group multiple items together under a single variable name.

Q2 DIFFERENCES

What is the primary conceptual difference between an array and a structure?

A. Arrays cannot be indexed while structures can
B. An array holds elements of the same type; a structure can hold members of different types
C. Structures cannot be declared as variables
D. Arrays always use the dot operator; structures use square brackets

Ans: B

Explanation: The core difference is homogeneity: arrays are same-type collections, structures may combine different types.

Q3 MEMORY ORGANIZATION

How does the memory organization of arrays compare to that of structures?

A. Arrays use non-contiguous memory; structures use contiguous memory
B. Neither uses contiguous memory
C. Both are stored only on the heap
D. Both typically use contiguous memory, but structure members may include padding for alignment

Ans: D

Explanation: Both are contiguous in memory, though structures may include extra padding bytes between members for alignment.

Q4 DATA STORAGE

How are elements typically retrieved in an array versus a structure?

A. Array elements are accessed by numeric index; structure members are accessed by name
B. Both are accessed only by name
C. Both are accessed only by numeric index
D. Array elements are accessed by name; structure members by index

Ans: A

Explanation: Arrays use an index like arr[i], while structures use named member access like s.member.

Q5 APPLICATIONS

Which is a more appropriate use case for a structure rather than an array?

A. Storing 100 integer scores of the same type
B. Storing a list of 50 identical float temperatures
C. Storing one employee's id, name, and salary together
D. Storing a sequence of characters as a string

Ans: C

Explanation: Combining different types (id, name, salary) for a single entity is a natural fit for a structure, not an array.

Q6 ADVANTAGES

Which is an advantage of a structure over an array for representing a record?

- A A structure is always smaller in memory than an equivalent array
- B A structure can combine different data types meaningfully into one named record
- C A structure automatically sorts its members
- D A structure cannot have more than one instance

Ans: B

Explanation: Structures can meaningfully combine dissimilar data types describing one entity, which arrays cannot do directly.

Q7 LIMITATIONS

Which is a limitation shared by both arrays and structures in standard C?

- A Neither can be passed to a function
- B Neither can contain more than one element/member
- C Neither can be used inside another structure
- D Neither automatically resizes itself once declared with a fixed size

Ans: D

Explanation: Both arrays and structures with fixed-size members have a size fixed at compile time and do not auto-resize.

Q8 SIMILARITIES

In what way are arrays and structures similar regarding functions?

- A Both can be passed to and returned from functions
- B Neither can ever be passed to a function
- C Only arrays can be returned from functions
- D Only structures can be passed to functions

Ans: A

Explanation: Both arrays (via pointers/decay) and structures (by value or pointer) can be used with functions.

Q9 DIFFERENCES

Which statement correctly contrasts declaring an array vs a structure?

- A int arr[5]; and struct Point p; both create heterogeneous collections
- B Declaring an array requires a tag; declaring a structure does not
- C int arr[5]; creates a homogeneous set of 5 ints; struct Point p; creates one variable with possibly different-typed members
- D Neither requires specifying a size or member list

Ans: C

Explanation: An array declaration fixes a count of same-type elements, while a structure defines named, possibly differently-typed members.

Q10 DATA STORAGE

Which best explains how data is stored differently between the two?

- A Arrays store named fields; structures store indexed elements
- B Arrays store a sequence of same-type values; structures store a set of named, possibly different-type values
- C Both store data identically with no difference
- D Structures cannot store more than one value ever

Ans: B

Explanation: Arrays are a sequence of same-type values; structures are a set of named fields that may differ in type.

Q11 APPLICATIONS

Which combination best illustrates using arrays and structures together?

A Using only arrays and avoiding structures entirely

B Using only structures and avoiding arrays entirely

C Never combining the two constructs

D An array of structures, such as 'struct Student class[30];', to store many records of the same structured type

Ans: D

Explanation: Combining both, an array of structures, is a very common way to store many records of one structured type.

Q12 LIMITATIONS

Which is a genuine limitation of arrays compared to structures?

A Arrays cannot naturally combine unrelated different-typed values into one meaningful unit

B Arrays can never be looped over

C Arrays are always heterogeneous

D Arrays cannot be declared with a fixed size

Ans: A

Explanation: Because arrays require uniform element types, they cannot naturally combine different-typed data the way structures can.

Functions and Structures

12 Multiple Choice Questions | With Answers & Explanations

Q1 USING STRUCTURES WITH FUNCTIONS

In C, structures can be used with functions in which of the following ways?

A. Only passed by value, never returned
B. Passed as arguments and also returned as the function's return type
C. Only passed by reference using arrays
D. Never allowed inside function bodies

Ans: B

Explanation: C allows structures to be passed to functions as arguments and also returned from functions as the return type.

Q2 PASSING STRUCTURES TO FUNCTIONS

When a structure is passed to a function by value, what happens?

A. The original structure variable is modified directly
B. Only a pointer to the structure is passed
C. Nothing is copied; the function accesses the caller's memory
D. A complete copy of the structure is made and passed to the function

Ans: D

Explanation: Passing by value copies the entire structure's contents into the function's local parameter.

Q3 RETURNING STRUCTURES

Can a C function return an entire structure as its return type?

A. Yes, a function can be declared to return a struct type directly
B. No, functions can only return basic data types
C. Only pointers to structures can ever be returned
D. Only using global variables, never a return statement

Ans: A

Explanation: C functions can be declared with a structure type as the return type and return a struct value directly.

Q4 APPLICATIONS

Which is a practical reason to pass a structure to a function?

A. To make the function unable to access any member
B. To force recompilation of the program
C. To let the function process an entire record (like a student) as one unit
D. To avoid using the dot operator inside the function

Ans: C

Explanation: Passing a structure lets a function operate on a whole related record in a single, clean parameter.

Q5 FUNCTION INTERACTION

What is a drawback of passing a large structure to a function by value?

A. It is always faster than passing a pointer
B. It creates overhead due to copying the entire structure's data
C. It automatically converts to a pointer, avoiding any copy
D. It prevents the function from accessing any members

Ans: B

Explanation: Passing large structures by value copies all their data, which can be costly compared to passing a pointer.

Q6 PASSING STRUCTURES TO FUNCTIONS

To avoid the overhead of copying a large structure when calling a function, it is better to:

A Pass each member individually as many separate arguments
B Avoid using structures in functions altogether
C Always pass by value regardless of size
D Pass the address of the structure (a pointer) to the function

Ans: D

Explanation: Passing a pointer avoids copying the whole structure, since only the address is passed.

Q7 RETURNING STRUCTURES

Consider 'struct Point makePoint(int a,int b){struct Point p={a,b}; return p;}'. What does this do?

A It creates a local structure and returns a copy of it to the caller
B It causes undefined behavior because local structures cannot be returned
C It returns only the address of p, not its value
D It modifies a global structure variable

Ans: A

Explanation: The function builds a local structure and returns a copy of its value, which is valid and common in C.

Q8 USING STRUCTURES WITH FUNCTIONS

Which function signature correctly accepts a structure by value?

A void show(struct Point *p);
B void show(int p);
C void show(struct Point p);
D void show(Point p);

Ans: C

Explanation: 'void show(struct Point p);' correctly declares a parameter of structure type, passed by value.

Q9 FUNCTION INTERACTION

If a function modifies a structure parameter passed by value, what happens to the caller's original structure?

A The caller's structure is also modified
B The caller's original structure remains unchanged, since only a copy was modified
C It causes a runtime crash
D Both the copy and original are deleted

Ans: B

Explanation: Since pass-by-value creates a separate copy, changes to the parameter inside the function do not affect the caller's original.

Q10 APPLICATIONS

Which is an appropriate real use of returning a structure from a function?

A A function that only prints text with no data
B A function that returns nothing (void)
C A function that takes no parameters and no output
D A function 'struct Date getCurrentDate()' that builds and returns a Date structure

Ans: D

Explanation: Returning a fully built structure, like a computed date, is a natural and common use of structure return types.

Q11 PASSING STRUCTURES TO FUNCTIONS

Which correctly passes a structure by reference (using a pointer) to a function?

A void update(struct Point *p) { p->x = 10; }

B void update(struct Point p) { p->x = 10; }

C void update(int p) { p.x = 10; }

D void update() { p.x = 10; }

Ans: A

Explanation: Declaring the parameter as a pointer, 'struct Point *p', and using -> allows modifying the caller's original structure.

Q12 FUNCTION INTERACTION

What must match between a function's return type declaration and its return statement, when returning a structure?

A Nothing needs to match

B Only the tag name needs to differ

C The structure type of the returned value must match the function's declared structure return type

D Only the first member needs to match

Ans: C

Explanation: The value returned must be of the exact structure type declared as the function's return type.

Nested Structures

13 Multiple Choice Questions | With Answers & Explanations

Q1 DEFINITION

What is a nested structure in C?

A. A structure that contains another structure as one of its members
B. A structure with no members at all
C. Two unrelated structures declared in the same file
D. A structure declared inside a function only

Ans: A

Explanation: A nested structure occurs when one structure type is used as a member inside another structure.

Q2 DECLARATION

Which correctly declares a nested structure, with 'Date' nested inside 'Employee'?

A. struct Employee { Date doj; };
B. struct Employee { struct Date; };
C. struct Employee { char name[20]; struct Date doj; };
D. struct Date { struct Employee e; }; struct Employee {};

Ans: C

Explanation: The inner structure type is referenced with 'struct Date doj;', correctly declaring a nested member with a name.

Q3 INITIALIZATION

How would you initialize a nested structure, e.g. Employee containing a Date member, at declaration?

A. It cannot be initialized directly; only assigned member-by-member after declaration
B. struct Employee e = {"Ravi", {12, 5, 2024}}; using nested braces
C. struct Employee e = "Ravi", 12, 5, 2024; without any braces
D. struct Employee e[] = {"Ravi"}; ignoring the Date entirely

Ans: B

Explanation: Nested braces are used within the outer initializer list to initialize the inner structure's members.

Q4 ACCESSING MEMBERS

How do you access the 'year' member of the nested Date structure inside Employee variable 'e'?

A. e.year
B. e.Date.year
C. year.e.doj
D. e.doj.year

Ans: D

Explanation: Chained dot operators are used: first reach the nested member 'doj', then its own member 'year': e.doj.year.

Q5 APPLICATIONS

Which is a good real-world use of a nested structure?

A. An Employee structure containing a nested Address structure with street, city, and pincode
B. A single int variable
C. An array with no structure involved
D. Two completely unrelated structures in different files

Ans: A

Explanation: Grouping an Address (street, city, pincode) as a nested structure inside Employee models a real-world relationship well.

Q6 ADVANTAGES

What is an advantage of using nested structures?

A They prevent any further member access
B They eliminate the need for the dot operator
C They allow logical grouping of related sub-records within a larger record
D They can only exist inside functions

Ans: C

Explanation: Nested structures let a large record be organized into logical sub-groups, improving clarity and structure.

Q7 DECLARATION

Which of the following is a valid rule for declaring nested structures?

A The inner structure can be declared after the outer structure that uses it
B The inner structure must be fully declared/defined before it is used as a member type in the outer structure
C Nested structures cannot have their own tag name
D Only one level of nesting is permitted in C

Ans: B

Explanation: In C, the inner structure type must already be defined before the outer structure can use it as a member's type.

Q8 ACCESSING MEMBERS

For deeply nested structures, how are the deepest members accessed?

A Only through pointers, never with dot notation
B It is not possible to access more than 2 levels deep
C Using array indexing only
D By chaining the dot operator through each level, e.g. a.b.c.d

Ans: D

Explanation: Multiple levels of nesting are accessed by chaining dot operators through each intermediate structure member.

Q9 USAGE

Which statement about nested structures and memory is TRUE?

A The outer structure's total size includes the memory required by its nested structure member(s)
B Nested structure members do not occupy any memory
C Nesting always doubles the memory used regardless of member sizes
D Nested structures share memory with unrelated variables

Ans: A

Explanation: A nested structure member contributes its own size to the total memory footprint of the outer structure.

Q10 DECLARATION

Can a structure contain a nested structure of its own exact same type directly as a value member?

A Yes, always, with no restriction
B Yes, but only for the first member
C No, this causes infinite recursive size and a compile error; only a pointer to the same type is allowed
D Yes, but only in nested nested pairs

Ans: C

Explanation: A structure cannot directly contain a value member of its own type (infinite size), but it can contain a pointer to its own type.

Q11 APPLICATIONS

Which nested structure design correctly represents a Company with multiple Departments, each having employees?

A Flattening everything into a single structure with no nesting

B struct Company { struct Department depts[10]; }; where struct Department { struct Employee emp[20]; };

C Declaring all employees as separate unrelated global variables

D Avoiding structures and using only arrays of int

Ans: B

Explanation: Nesting arrays of structures within structures models hierarchical real-world data like Company-Department-Employee.

Q12 INITIALIZATION

Which correctly performs partial initialization of a nested structure, leaving the inner Date defaulted to zero?

A struct Employee e = ; (nothing at all)

B struct Employee e = {12, 5, 2024, "Ravi"}; (wrong order, no nested braces)

C It is impossible to partially initialize nested structures

D struct Employee e = {"Ravi"}; leaving the nested doj structure's members zero-initialized

Ans: D

Explanation: Just as with simple structures, omitted nested members (and their own members) are automatically zero-initialized.

Q13 DEFINITION

Which best distinguishes a nested structure from simply having two separate, unrelated structures?

A In a nested structure, one structure type is embedded as an actual member inside another, forming one combined type

B There is no real difference between the two

C Nested structures cannot be given tags

D Nested structures must always be arrays

Ans: A

Explanation: Nesting specifically means one structure type becomes a member of another, combining them into a single composite type.

Anonymous Structures

9 Multiple Choice Questions | With Answers & Explanations

Q1 CONCEPT

What is an anonymous structure in C?

A. A structure with multiple tags
B. A structure declared only inside a loop
C. A structure defined without giving it a tag name
D. A structure that can never have members

Ans: C

Explanation: An anonymous structure is one defined without a tag name, often used directly with a variable or as an unnamed nested member.

Q2 DECLARATION

Which of the following declares an anonymous structure variable?

A. struct Point { int x, y; };
B. struct { int x, y; } p;
C. struct Point p;
D. typedef struct Point p;

Ans: B

Explanation: Omitting the tag name while still declaring a variable, as in 'struct { int x, y; } p;', creates an anonymous structure.

Q3 USAGE

Since an anonymous structure has no tag, how can it typically be reused elsewhere in standard C?

A. It can be reused freely by its tag name anywhere
B. It automatically becomes a global type usable everywhere
C. It converts itself into a named structure automatically
D. It generally cannot be reused by name; typedef is often used to give it a reusable alias

Ans: D

Explanation: Without a tag, an anonymous structure can't be referenced later by name; typedef is commonly used to give it a reusable alias.

Q4 CONCEPT

An anonymous structure is most commonly combined with which keyword to create a reusable named type?

A. typedef
B. extern
C. static
D. register

Ans: A

Explanation: 'typedef struct { ... } TypeName;' combines an anonymous structure with typedef to create a reusable alias.

Q5 CHARACTERISTICS

Which is a characteristic of an anonymous structure used as a nested member (compiler-supported)?

A. It requires a member name to access its own fields from the outer structure
B. It cannot contain any members
C. Its members can sometimes be accessed directly through the outer structure without an intermediate name
D. It must always be declared globally

Ans: C

Explanation: In compilers supporting anonymous nested structures/unions, inner members can sometimes be accessed directly via the outer structure.

Q6 APPLICATIONS

Which is a typical use case for an anonymous structure with typedef?

- A Declaring a function prototype
- B typedef struct { int x, y; } Point; to create a concise, reusable Point type without repeating 'struct'
- C Declaring a for loop counter
- D Declaring a preprocessor macro

Ans: B

> **Explanation:** This pattern lets programmers use 'Point' directly as a type name afterward, without needing to write 'struct' each time.

Q7 DECLARATION

Which statement about an anonymous structure declaration is TRUE?

- A It must always have exactly one member
- B It cannot be initialized
- C It always requires a separate tag declared elsewhere
- D It omits the tag name directly after the 'struct' keyword

Ans: D

> **Explanation:** An anonymous structure simply omits the tag identifier that would normally follow the 'struct' keyword.

Q8 USAGE

What is a limitation of using anonymous structures without typedef?

- A Variables of that exact structure layout cannot easily be declared again elsewhere by referring to a type name
- B They cannot have any members
- C They cannot be used with the dot operator
- D They must always be arrays

Ans: A

> **Explanation:** Without a tag or typedef alias, there's no name to use for declaring further variables of that same structure layout.

Q9 CHARACTERISTICS

Compared to a tagged structure, an anonymous structure primarily differs in that it:

- A Occupies different memory layout rules entirely
- B Cannot store any data
- C Lacks a tag identifier, though its members and behavior otherwise work the same way
- D Must be declared using the union keyword instead

Ans: C

> **Explanation:** An anonymous structure behaves like a normal structure in memory and member access; it simply has no tag name.

Self-Referential Structures

10 Multiple Choice Questions | With Answers & Explanations

Q1 DEFINITION

What is a self-referential structure?

A. A structure that contains a copy of itself as a value member
B. A structure that contains a pointer to another structure variable of its own type
C. A structure that has no members
D. A structure that can only hold integers

Ans: B

Explanation: A self-referential structure contains a pointer member pointing to another instance of the same structure type.

Q2 CONCEPT

Why must a self-referential structure use a pointer to itself rather than a value member of itself?

A. Pointers are always faster than any other data type
B. C does not allow pointers inside structures
C. It is simply a stylistic choice with no technical reason
D. A value member of the same type would require infinite memory, since its size would depend on itself

Ans: D

Explanation: A value member of the same struct type would create an unresolvable, infinitely recursive size, so only a pointer (fixed size) works.

Q3 DECLARATION

Which declaration correctly creates a self-referential structure for a singly linked list node?

A. struct Node { int data; struct Node *next; };
B. struct Node { int data; struct Node next; };
C. struct Node { int data; Node *next; };
D. struct Node { int data; struct Node *Node; };

Ans: A

Explanation: 'struct Node *next;' correctly declares a pointer member of the same structure type, forming a valid self-referential structure.

Q4 POINTER MEMBERS

In a self-referential structure like a linked list node, what does the 'next' pointer member typically store?

A. The value of the current node's data member
B. A separate array of all nodes
C. The memory address of the next node in the sequence, or NULL if there is none
D. A count of the total number of nodes

Ans: C

Explanation: The 'next' pointer holds the address of the next node in the list, and is set to NULL to mark the end of the list.

Q5 APPLICATIONS

Self-referential structures are the foundational building block for which common data structure?

A. Simple arrays only
B. Linked lists (and similarly trees)
C. Two-dimensional arrays only
D. Basic integer variables

Ans: B

Explanation: Self-referential structures, with pointers to their own type, form the basis of linked lists and tree structures.

Q6 CHARACTERISTICS

Which characteristic is true of a self-referential structure?

A It must have exactly one member, the pointer itself
B It cannot contain any data members other than the pointer
C It always requires dynamic memory allocation to even be declared
D It combines regular data members with at least one pointer member of its own structure type

Ans: D

Explanation: A typical self-referential structure has ordinary data fields plus one or more pointer members referencing the same type.

Q7 CONCEPT

What does it mean when a self-referential structure's pointer member is set to NULL?

A It indicates there is no further connected node (e.g., the end of a linked list)
B It means the structure has crashed
C It means the pointer now points to itself
D It automatically deletes the structure

Ans: A

Explanation: A NULL pointer in a self-referential structure conventionally signals there is no next (or connected) element.

Q8 DECLARATION

Why is 'struct Node *next;' valid even though 'struct Node' is still being defined at that point?

A It is actually invalid and never compiles
B The compiler ignores incomplete types entirely
C A pointer to an incomplete type is allowed because pointers have a fixed, known size regardless of the pointed-to type
D 'next' must be declared outside the structure instead

Ans: C

Explanation: Since all pointers have a fixed size, the compiler can allocate space for a pointer member even while the full type is still being defined.

Q9 APPLICATIONS

Which of these best illustrates a practical use of a self-referential structure?

A A structure storing only two int coordinates for a point
B A tree node structure with data, a left child pointer, and a right child pointer, both of the node's own type
C A structure with only a single char member
D An array of 10 unrelated integers

Ans: B

Explanation: A binary tree node, referencing left and right children of its own type via pointers, is a classic self-referential structure.

Q10 POINTER MEMBERS

How many self-referencing pointer members can a self-referential structure have?

A Exactly one, always
B Zero, by definition
C At most two, and no more
D One or more, depending on the design (e.g., a tree node may have two: left and right)

Ans: D

Explanation: A self-referential structure can have one or multiple pointer members of its own type, as needed by the data structure's design.

Pointer as Member of Structure

10 Multiple Choice Questions | With Answers & Explanations

Q1 POINTER MEMBERS

Which of the following correctly declares a structure with a pointer member pointing to an int?

A struct Data { int *ptr; };
B struct Data { int ptr*; };
C struct Data { *int ptr; };
D struct Data { ptr int*; };

Ans: A

Explanation: 'int *ptr;' correctly declares a pointer-to-int member named ptr inside the structure.

Q2 DECLARATION

Can a structure member be a pointer to a different structure type entirely (not itself)?

A No, pointer members can only point to basic types
B No, pointer members are never allowed inside structures
C Yes, a structure member can be a pointer to any valid type, including another structure type
D Only if the pointed-to structure has no members

Ans: C

Explanation: A structure can have pointer members pointing to basic types, other structures, functions, or its own type.

Q3 ACCESS

Given 'struct Data { int *ptr; } d; int x = 5; d.ptr = &x;', how do you access the value pointed to by d.ptr?

A d.ptr
B *(d.ptr)
C d->ptr
D &d.ptr

Ans: B

Explanation: '*(d.ptr)' dereferences the pointer member to get the int value it points to.

Q4 APPLICATIONS

What is a practical use of a pointer member inside a structure?

A It always wastes memory with no benefit
B It prevents the structure from ever being used with functions
C It forces the structure to only store integers
D It allows dynamic linking to other data, like a string stored via a char pointer instead of a fixed array

Ans: D

Explanation: A pointer member (e.g., char *name;) lets the structure reference dynamically allocated or external data flexibly.

Q5 USAGE

Which is TRUE about using 'char *name;' versus 'char name[20];' as a structure member?

A The pointer version requires separate memory allocation (e.g., malloc) before use; the array version reserves fixed space directly
B Both behave identically with no differences
C The pointer version always uses less total memory in every case
D The array version cannot ever store a string

Ans: A

Explanation: A char pointer member needs memory to be allocated (e.g., via malloc) before storing data, unlike a fixed-size array member.

Q6 POINTER MEMBERS

If a structure has a member 'struct Node *next;' where Node is the structure's own type, what kind of pointer member is this?

A A function pointer member
B A void pointer member
C A self-referential pointer member
D An array pointer member

Ans: C

Explanation: A pointer member referring to the structure's own type is specifically called a self-referential pointer member.

Q7 ACCESS

For 'struct Emp { char *name; } e; e.name = "Ravi";', which correctly prints the name?

A printf("%s", e.name[0]);
B printf("%s", e.name);
C printf("%s", &e);
D printf("%s", e);

Ans: B

Explanation: Since name is a char pointer, printf with %s and e.name directly prints the string it points to.

Q8 DECLARATION

Which of these correctly declares a structure member that is a pointer to a function?

A struct Ops { int func(); };
B struct Ops { func *op; };
C struct Ops { int op(); };
D struct Ops { int (*op)(int, int); };

Ans: D

Explanation: 'int (*op)(int, int);' correctly declares a member that is a pointer to a function taking two ints and returning int.

Q9 APPLICATIONS

A structure member that is a pointer to another structure type is often used to model which kind of relationship?

A A relationship or link between two different records, such as an Employee pointing to its Department
B No relationship at all
C Only mathematical relationships between integers
D It can only be used for self-reference, never for other types

Ans: A

Explanation: A pointer member to another structure type is commonly used to model a link or relationship between two different records.

Q10 USAGE

What must be done before dereferencing a pointer member of a structure that was declared but not assigned an address?

A Nothing; it is always safe to dereference immediately
B The structure itself must be deleted first
C The pointer member must first be assigned a valid address (e.g., via & or malloc) before it is dereferenced
D Only global structures need this step

Ans: C

Explanation: Dereferencing an uninitialized pointer member leads to undefined behavior; it must first point to valid, allocated memory.

Pointer to Structure

13 Multiple Choice Questions | With Answers & Explanations

Q1 DECLARATION

Which correctly declares a pointer to a structure of type 'struct Point'?

A struct Point ptr;
B struct Point *ptr;
C struct *Point ptr;
D Point struct *ptr;

Ans: B

Explanation: 'struct Point *ptr;' declares ptr as a pointer variable that can store the address of a struct Point variable.

Q2 INITIALIZATION

Given 'struct Point p = {1,2};', which correctly initializes a pointer to point to p?

A struct Point *ptr = p;
B struct Point *ptr = *p;
C struct Point ptr = &p;
D struct Point *ptr = &p;

Ans: D

Explanation: The address-of operator (&) is required to assign p's address to the pointer: struct Point *ptr = &p;.

Q3 ACCESSING MEMBERS

Given 'struct Point *ptr = &p;', which correctly accesses member x through the pointer?

A ptr->x
B ptr.x
C *ptr.x
D x->ptr

Ans: A

Explanation: The arrow operator (->) is the standard way to access a member through a structure pointer: ptr->x.

Q4 ACCESSING MEMBERS

Which two expressions are equivalent for accessing member x via pointer ptr?

A ptr->x and ptr.x
B ptr->x and &ptr.x
C ptr->x and (*ptr).x
D ptr->x and *ptr->x

Ans: C

Explanation: 'ptr->x' is exactly equivalent to '(*ptr).x'; both dereference the pointer first, then access the member.

Q5 POINTER VARIABLES

What does a structure pointer variable actually store?

A A full copy of the entire structure's data
B The memory address of a structure variable
C Only the first member's value
D Nothing until the program ends

Ans: B

Explanation: Like any pointer, a structure pointer stores the memory address of the structure variable it points to, not a copy.

Q6 APPLICATIONS

Why are pointers to structures commonly used when passing large structures to functions?

A They make the code harder to read with no benefit
B They force a complete copy of the structure regardless
C They are required by the C standard for all structures
D They avoid the overhead of copying the whole structure by passing just the address

Ans: D

Explanation: Passing a pointer avoids copying potentially large structure data, improving efficiency.

Q7 DECLARATION

Which of these declares an array of pointers to 'struct Node' structures?

A struct Node *arr[10];
B struct Node arr[10]*;
C struct *Node arr[10];
D struct Node *arr(10);

Ans: A

Explanation: 'struct Node *arr[10];' declares an array of 10 pointers, each capable of pointing to a struct Node.

Q8 INITIALIZATION

If 'struct Point *ptr;' is declared but never assigned an address, what does ptr contain?

A Automatically the address of a zero-initialized structure
B The value 0 guaranteed by the C standard for local pointers
C An indeterminate (garbage) address, since it is an uninitialized local pointer
D The address of the main function

Ans: C

Explanation: An uninitialized local pointer holds an indeterminate value; dereferencing it leads to undefined behavior.

Q9 ACCESSING MEMBERS

Given 'struct Emp *e; e = &emp1;', which correctly sets the 'salary' member to 50000 through the pointer?

A e.salary = 50000;
B e->salary = 50000;
C *e.salary = 50000;
D salary->e = 50000;

Ans: B

Explanation: The arrow operator correctly accesses and assigns a value to a member through a structure pointer: e->salary = 50000;.

Q10 APPLICATIONS

Structure pointers are essential for implementing which of the following?

A Only simple arithmetic expressions
B Static global constants only
C Basic for loops with no data structures
D Dynamic data structures such as linked lists and trees, where nodes reference each other via pointers

Ans: D

Explanation: Structure pointers allow nodes to reference one another, forming the basis of dynamic structures like linked lists and trees.

Q11 POINTER VARIABLES

Which statement about pointer variables to structures is TRUE?

- **A** A single pointer variable can, over time, point to different structure variables of the same type
- **B** A pointer to a structure can only ever point to one fixed variable for the life of the program
- **C** Structure pointers cannot be reassigned once set
- **D** Structure pointers can only point to global variables

Ans: A

> **Explanation:** Like any pointer, a structure pointer can be reassigned at different times to point to different variables of the same type.

Q12 DECLARATION

Can a structure contain a pointer to another dynamically allocated structure of the same type using malloc?

- **A** No, malloc cannot be used with structures at all
- **B** No, only static structures are allowed
- **C** Yes, e.g. struct Node *next = (struct Node*) malloc(sizeof(struct Node));
- **D** Yes, but only for the first structure ever created in the program

Ans: C

> **Explanation:** malloc can dynamically allocate memory for a new structure, and its address can be stored in a pointer member.

Q13 ACCESSING MEMBERS

Given a pointer 'struct Book *bptr' pointing to a Book structure with a char array title[30], which correctly accesses the title?

- **A** bptr.title
- **B** bptr->title
- **C** title->bptr
- **D** *bptr.title[0]

Ans: B

> **Explanation:** The arrow operator correctly accesses the title array member through the structure pointer: bptr->title.

Member Selector Operator (->)

12 Multiple Choice Questions | With Answers & Explanations

Q1 PURPOSE

What is the primary purpose of the member selector operator (->)?

A To declare a new structure
B To allocate memory
C To compare two structures
D To access a structure's member through a pointer to that structure

Ans: D

Explanation: The -> operator is specifically used to access members of a structure through a pointer to that structure.

Q2 SYNTAX

What is the correct syntax for using the member selector operator?

A pointer->member
B member->pointer
C pointer.->member
D pointer<-member

Ans: A

Explanation: The correct syntax is 'pointer->member', where pointer is a variable pointing to a structure.

Q3 ACCESSING MEMBERS THROUGH POINTERS

Given 'struct Point *p = &pt;', which correctly accesses and prints member y?

A printf("%d", p.y);
B printf("%d", *p.y);
C printf("%d", p->y);
D printf("%d", y->p);

Ans: C

Explanation: 'p->y' correctly accesses member y of the structure pointed to by pointer p.

Q4 APPLICATIONS

The -> operator is especially useful in which programming scenario?

A Declaring simple int variables
B Traversing linked lists, where each node's members are accessed through a pointer, e.g. head->next
C Performing string concatenation
D Declaring function prototypes

Ans: B

Explanation: Linked list traversal relies heavily on -> to access data and next-node pointers through each node's pointer.

Q5 COMPARISON WITH . OPERATOR

When should you use -> instead of . to access a structure's member?

A Only when the structure has more than 5 members
B Never; . and -> are always interchangeable in every context
C Only for structures containing arrays
D When you have a pointer to the structure rather than the structure variable itself

Ans: D

Explanation: Use -> when working through a pointer to a structure; use . when working directly with a structure variable.

Q6 SYNTAX

Given 'struct Emp *e; e->salary = e->salary + 1000;', what does this statement do?

A It increases the salary member (accessed through pointer e) by 1000
B It creates a new Emp structure
C It causes a syntax error since -> cannot appear twice
D It decreases the salary member by 1000

Ans: A

Explanation: This reads the current salary through the pointer, adds 1000, and stores it back, all via the -> operator.

Q7 PURPOSE

Why can't the dot (.) operator be used directly in place of -> when you have a structure pointer?

A Dot and arrow always produce identical results with pointers
B C forbids the dot operator entirely
C The dot operator expects a structure variable, not an address; applying it to a pointer type is a type mismatch
D Only because of a stylistic convention with no real technical reason

Ans: C

Explanation: The dot operator is defined to work on structure variables directly; using it on a pointer without dereferencing is a type error.

Q8 ACCESSING MEMBERS THROUGH POINTERS

For 'struct Node *head; head->data = 10; head->next = NULL;', what is being set up?

A Two separate unrelated structures
B The data and next members of the node pointed to by head, typical of initializing a linked list node
C A structure array
D A function pointer

Ans: B

Explanation: This code sets both the data value and the next pointer (to NULL) of a single node pointed to by head.

Q9 APPLICATIONS

Which of the following demonstrates a valid, real use of the -> operator?

A int x -> 5; assigning a value to a plain int
B struct Emp e; e -> name; (e is not a pointer here)
C 5 -> struct Point p;
D struct Emp *e = &emp1; printf("%s", e->name);

Ans: D

Explanation: Here, e is a genuine structure pointer, and -> correctly accesses the name member it points to.

Q10 COMPARISON WITH . OPERATOR

Which correctly explains the relationship between p->x and (*p).x?

A They are exactly equivalent; -> is essentially shorthand for dereference-then-dot-access
B They always produce different results
C (*p).x is invalid syntax in C
D p->x only works with arrays, not structures

Ans: A

Explanation: 'p->x' is defined as shorthand for '(*p).x'; both dereference the pointer and access the member.

Q11 PURPOSE

Which best describes what happens internally when you write 'p->member'?

- A The compiler ignores the pointer and reads member from a fixed address
- B It first multiplies p by member
- C The compiler dereferences pointer p to get the structure, then accesses 'member' within it
- D It always triggers a memory allocation

Ans: C

> **Explanation:** Internally, p->member first follows the pointer (dereference) to the structure, then retrieves the specified member.

Q12 SYNTAX

Can the arrow operator be chained, as in 'emp->dept->name', when dept is itself a pointer member?

- A No, chaining -> is never allowed in C
- B Yes, if dept is a pointer to another structure containing a 'name' member, chaining -> is valid
- C Only if name is an integer
- D Only inside a for loop

Ans: B

> **Explanation:** When a member is itself a pointer to another structure, -> can be chained to reach members of that nested structure.

Comparison Between Indirection (*) Operator and Member Selector (->) Operator

10 Multiple Choice Questions | With Answers & Explanations

Q1 PURPOSE

What is the purpose of the indirection (*) operator in the context of structures?

A To declare structure tags
B To access array elements only
C To compare two pointers
D To dereference a pointer, obtaining the structure (or value) it points to

Ans: D

Explanation: The * operator dereferences a pointer, giving access to the actual structure (or value) that the pointer refers to.

Q2 SYNTAX

Which expression correctly uses the indirection operator (with dot) to access member x via pointer p?

A (*p).x
B *p.x (without parentheses)
C p*.x
D x.*p

Ans: A

Explanation: Parentheses are required: '(*p).x' first dereferences p, then accesses member x; without them, precedence causes errors.

Q3 USAGE

Why are parentheses necessary in '(*p).x' but not in 'p->x'?

A Parentheses are never actually necessary in either form
B Because -> has lower precedence than *
C Because '.' has higher precedence than '*', so without parentheses '*p.x' would incorrectly evaluate p.x first
D Because '.' cannot be used with pointers under any circumstance

Ans: C

Explanation: The dot operator binds tighter than *, so without parentheses, 'p.x' would be attempted before dereferencing, causing an error.

Q4 DIFFERENCES

What is the key difference between using * (with .) and -> to access a structure member through a pointer?

A * can only be used with arrays, -> only with structures
B (*p).x and p->x achieve the identical result, but -> is more concise and commonly preferred
C * always fails to compile with structures
D -> can only be used once per program

Ans: B

Explanation: Both forms achieve the same member access, but -> is the more concise, standard, and preferred syntax in practice.

Q5 EXAMPLES

Given 'struct Point *p = &pt;', which pair of statements produce the same result?

A p->x = 5; and p.x = 5;
B (*p).x = 5; and p[x] = 5;
C p->x = 5; and *p = 5;
D p->x = 5; and (*p).x = 5;

Ans: D

Explanation: Both 'p->x = 5;' and '(*p).x = 5;' assign 5 to the same member x of the structure pointed to by p.

Q6 APPLICATIONS

In modern C code, which form is generally preferred for readability when accessing structure members via pointers?

A The -> operator, e.g. p->x, since it is shorter and clearer

B The (*p).x form, since it is always faster at runtime

C Neither form is ever used in practice

D Both are equally rare and discouraged

Ans: A

Explanation: The -> operator is generally preferred in practice because it is shorter and more readable than the equivalent (*p).x form.

Q7 USAGE

Which of the following is TRUE regarding operator precedence relevant to * and ->?

A -> has lower precedence than most arithmetic operators

B * always has higher precedence than -> in every expression

C -> operator already includes the dereference step, so no separate * or parentheses are needed

D -> cannot be used in the same expression as any other operator

Ans: C

Explanation: The -> operator combines dereferencing and member access into a single operation, unlike using * separately with dot.

Q8 EXAMPLES

Which of these expressions is INVALID or would need correction, given 'struct Point *p;'?

A p->x

B *p.x (without parentheses, problematic since it doesn't do what's intended)

C (*p).x

D p->y

Ans: B

Explanation: Without parentheses, '*p.x' is parsed as '*(p.x)', which is invalid since p.x needs p to be a structure, not a pointer.

Q9 DIFFERENCES

Which best summarizes the difference in intent between * and -> when working with structure pointers?

A * is used to declare arrays; -> is used to declare structures

B Both operators serve completely unrelated, unconnected purposes

C * is exclusive to functions; -> is exclusive to arrays

D * generically dereferences any pointer; -> is specialized, combining dereference and member access for structures/unions

Ans: D

Explanation: * is a general dereference operator for any pointer type, while -> is specifically designed for accessing structure/union members via a pointer.

Q10 PURPOSE

Which code snippet correctly demonstrates equivalent use of both operators for the same task?

A int a = (*p).x; int b = p->x; // a and b will hold the same value

B int a = *p; int b = p->x; // these are unrelated, not equivalent

C int a = p.x; int b = p->x; // p.x is invalid since p is a pointer

D int a = (*p); int b = (*p).y; // not equivalent to each other

Ans: A

Explanation: Both '(*p).x' and 'p->x' access the same member x of the structure pointed to by p, yielding the same value.

Structure as Function Argument

11 Multiple Choice Questions | With Answers & Explanations

Q1 PASSING STRUCTURES BY VALUE

When a structure is used as a function argument passed by value, the function receives:

A The original structure's address only
B Nothing; structures cannot be arguments
C A separate copy of the entire structure's data
D Only the first member's value

Ans: C

Explanation: Passing by value copies the complete structure data into the function's local parameter.

Q2 FUNCTION PARAMETERS

Which function declaration correctly accepts a structure as an argument by value?

A void display(struct Point *p);
B void display(struct Point p);
C void display(int p);
D void display(Point p);

Ans: B

Explanation: 'void display(struct Point p);' declares a parameter of structure type, received by value (a copy).

Q3 APPLICATIONS

Why might a programmer choose to pass a structure as a function argument by value rather than by pointer?

A To always guarantee the fastest possible performance
B To force the function to modify the caller's original data
C It is required by the C standard in every case
D To ensure the function cannot accidentally modify the caller's original structure, since it works on a copy

Ans: D

Explanation: Passing by value protects the caller's original data, since the function can only modify its own local copy.

Q4 PASSING STRUCTURES BY VALUE

If 'void update(struct Point p){p.x=100;}' is called with 'update(pt);', what happens to pt.x after the call?

A pt.x remains unchanged, since only the copy p was modified
B pt.x also becomes 100
C The program crashes
D pt is deleted after the call

Ans: A

Explanation: Since p is a separate copy (pass by value), modifying p.x has no effect on the original pt.x.

Q5 PASSING STRUCTURES THROUGH FUNCTIONS

Which is a disadvantage of passing very large structures as function arguments by value repeatedly in a loop?

A It has no disadvantage whatsoever
B It automatically converts to pointer passing to optimize itself
C Repeated copying of large data can significantly reduce performance
D It prevents the function from being called more than once

Ans: C

Explanation: Copying large amounts of data on every call adds overhead, which can noticeably slow down performance in loops.

Q6 FUNCTION PARAMETERS

Which function correctly accepts a structure argument by reference (pointer) to allow modification of the caller's data?

A void update(struct Point p) { p.x = 5; }
B void update(struct Point *p) { p->x = 5; }
C void update(int *p) { p->x = 5; }
D void update() { p->x = 5; }

Ans: B

Explanation: Accepting a pointer parameter and using -> allows the function to directly modify the caller's original structure.

Q7 APPLICATIONS

Which is a valid real-world reason to pass a structure argument to a function?

A To make the function ignore the structure entirely
B To force the program to stop compiling
C To rename the structure's tag
D To let a function like 'float calculateArea(struct Rect r)' compute a result using the structure's fields

Ans: D

Explanation: Passing the structure lets the function directly use its fields, like length and width, to compute a result.

Q8 PASSING STRUCTURES THROUGH FUNCTIONS

Which correctly demonstrates passing a structure argument and reading (not modifying) it inside the function?

A void show(struct Point p) { printf("%d %d", p.x, p.y); }
B void show(struct Point p) { printf("%d %d", p->x, p->y); }
C void show(struct Point *p) { printf("%d %d", p.x, p.y); }
D void show() { printf("%d %d", p.x, p.y); }

Ans: A

Explanation: Since p is a value parameter (not a pointer), the dot operator correctly reads its members: p.x, p.y.

Q9 FUNCTION PARAMETERS

What must the type of the argument passed match, when calling a function expecting a structure parameter?

A It can be any unrelated data type
B Only the tag name needs to be similar in spelling
C The argument must be of the exact same structure type as declared in the function's parameter
D It must always be a pointer, regardless of the parameter's declared type

Ans: C

Explanation: The argument passed to a function must match the exact structure type expected by the function's parameter.

Q10 PASSING STRUCTURES BY VALUE

Passing a structure by value to a function is analogous to which behavior of basic data types like int?

A It behaves like passing an uninitialized variable
B Just like passing a plain int by value, the function receives its own independent copy of the data
C It behaves like a global variable shared by all functions
D It behaves exactly like passing an array, which decays to a pointer

Ans: B

Explanation: Structures passed by value follow the same copy semantics as basic types like int passed by value.

Q11 APPLICATIONS

Which scenario best justifies passing a structure by pointer instead of by value as a function argument?

A The structure has only a single small int member

B The function never needs to see any data from the structure

C The structure is never modified and is very small

D The structure is large, and/or the function needs to modify the caller's original data

Ans: D

Explanation: Pointers avoid copy overhead for large structures and are necessary when the function must modify the caller's original data.

Structure as Function Return Type

10 Multiple Choice Questions | With Answers & Explanations

Q1 RETURNING STRUCTURES

Which of the following correctly declares a function that returns a structure of type 'struct Point'?

A. struct Point getPoint();
B. void getPoint(struct Point);
C. int getPoint(struct Point);
D. struct Point *getPoint(void);

Ans: A

Explanation: 'struct Point getPoint();' declares a function whose return type is the structure itself, not a pointer to it.

Q2 SYNTAX

Inside a function declared to return 'struct Point', which statement correctly returns a structure value?

A. return &p; // returns a pointer, mismatched with the declared return type
B. return p.x;
C. return p; // where p is a local struct Point variable
D. return *p;

Ans: C

Explanation: 'return p;' correctly returns the entire structure value p, matching the function's declared struct Point return type.

Q3 APPLICATIONS

Which is a good example of a function that returns a structure?

A. int add(int a, int b);
B. struct Point addPoints(struct Point p1, struct Point p2); returning a combined Point
C. void printPoint(struct Point p);
D. char getChar();

Ans: B

Explanation: A function combining two points and returning the resulting Point structure is a classic use of structure return types.

Q4 ADVANTAGES

What is an advantage of a function returning a structure instead of using multiple output parameters?

A. It always executes faster with no exceptions
B. It prevents the function from using any local variables
C. It requires the caller to pass extra pointers regardless
D. It lets a function return several related values (the structure's members) together as one clean unit

Ans: D

Explanation: Returning a structure lets several related values be handed back to the caller together, in one clean unit.

Q5 PROGRAM BEHAVIOR

When a function returns a structure by value, what happens to the local structure variable after it returns?

A. Its value is copied out to the caller before the function's local memory is reclaimed
B. It continues to exist and can still be accessed directly by the caller using its original name
C. It automatically becomes a global variable
D. The program always crashes at that point

Ans: A

Explanation: The structure's value is copied into the caller's context, and the function's local storage is then reclaimed.

Q6 SYNTAX

Which is the correct way to store the structure returned by a function into a variable?

A struct Point p; p->getPoint();
B getPoint() = p;
C struct Point p = getPoint();
D struct Point p; getPoint(p);

Ans: C

Explanation: The returned structure value is assigned directly to a variable of the same structure type: struct Point p = getPoint();.

Q7 RETURNING STRUCTURES

Can a function that returns a structure also accept structure parameters at the same time?

A No, a function cannot both take and return structures
B Yes, a function can take structure parameters and also return a structure, e.g. struct Point add(struct Point a, struct Point b)
C Only if the parameters and return type are of different structure types
D Only if there are no parameters at all

Ans: B

Explanation: A function can freely accept one or more structure parameters and still return a structure as its result.

Q8 APPLICATIONS

Which is a practical application of a function returning a structure?

A A function that returns nothing (void) and takes no arguments
B A function that only prints characters
C A function that only performs file operations
D A function 'struct Complex addComplex(struct Complex a, struct Complex b)' returning the sum of two complex numbers

Ans: D

Explanation: Returning a Complex structure representing the sum is a natural application of structure return types in numeric computation.

Q9 PROGRAM BEHAVIOR

If 'struct Point getOrigin(){struct Point p={0,0}; return p;}' is called as 'struct Point o=getOrigin();', what is o.x?

A 0
B An indeterminate garbage value
C It causes a compile-time error
D 1

Ans: A

Explanation: The function returns a Point initialized to {0,0}, so o.x correctly receives the value 0 after assignment.

Q10 ADVANTAGES

Compared to using global variables to share structure data, returning a structure from a function is advantageous because it:

A Requires no function call at all
B Makes the data accessible from every part of the program automatically
C Keeps data local and explicit, improving modularity and reducing unintended side effects
D Eliminates the need for structures entirely

Ans: C

Explanation: Returning a structure keeps data flow explicit and local to the function call, which is cleaner than relying on global state.